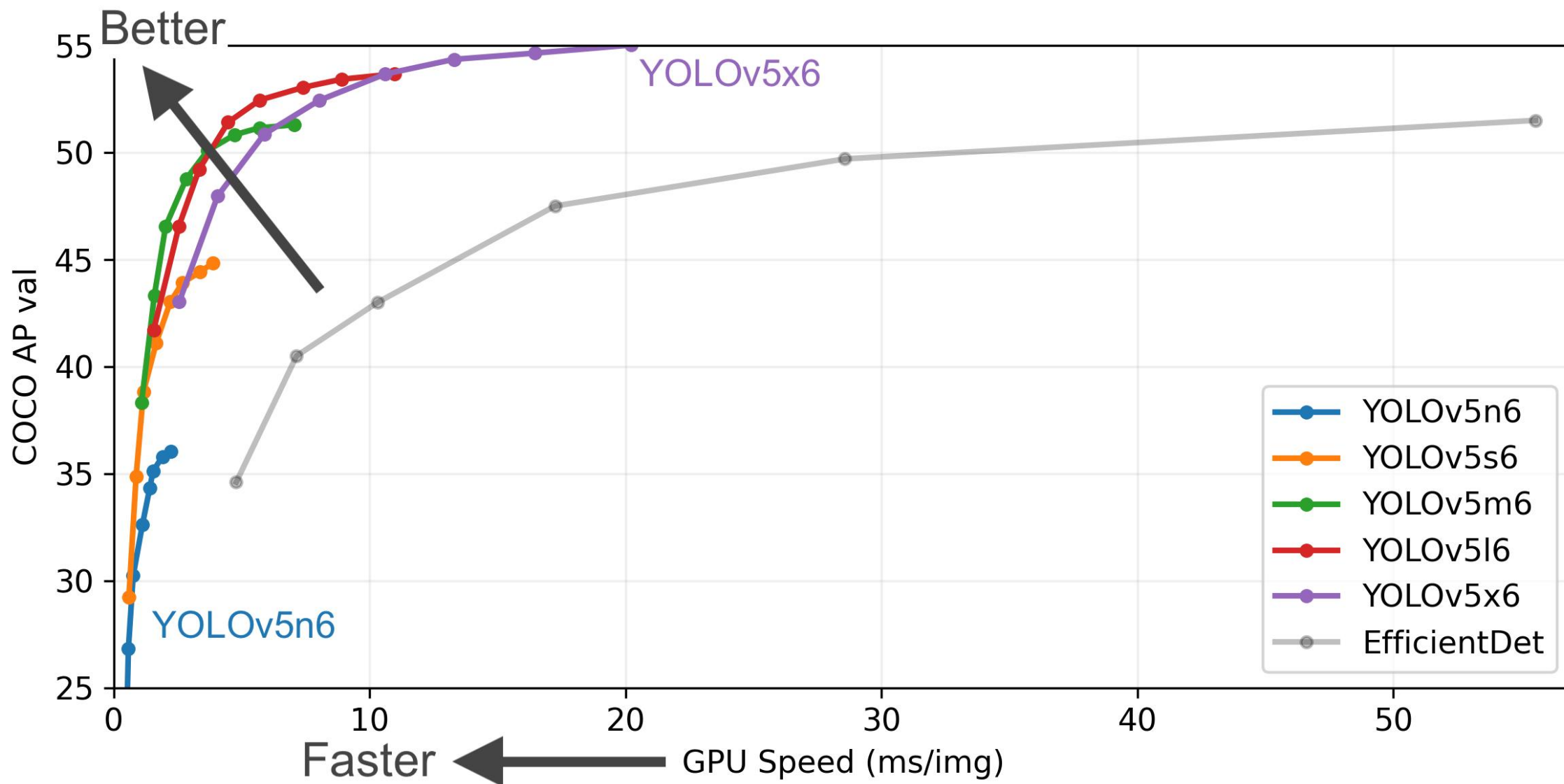




CV项目

YOLOv5

YOLO v5

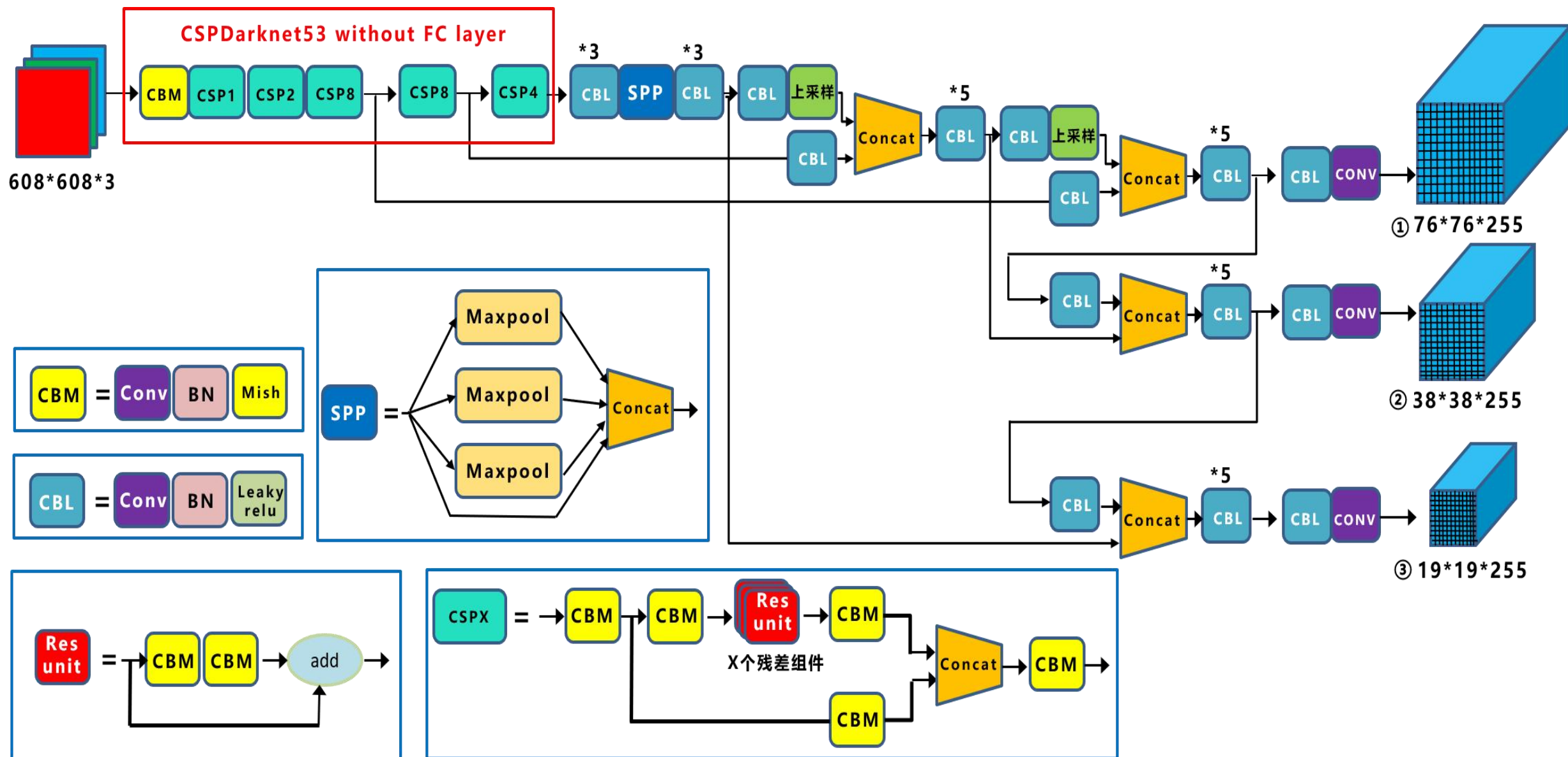


YOLO v5

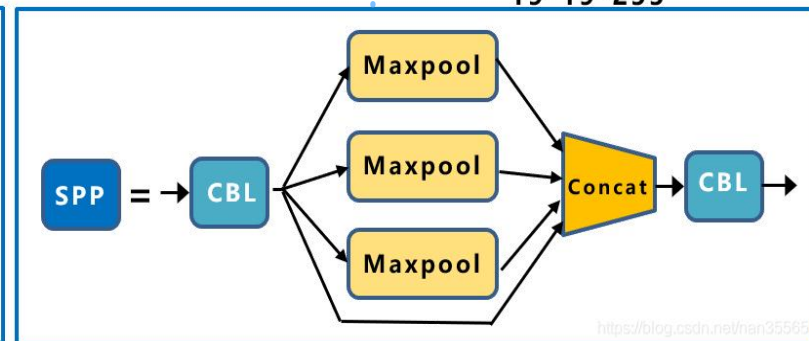
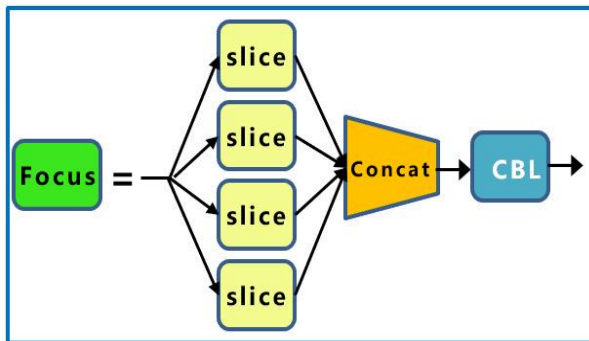
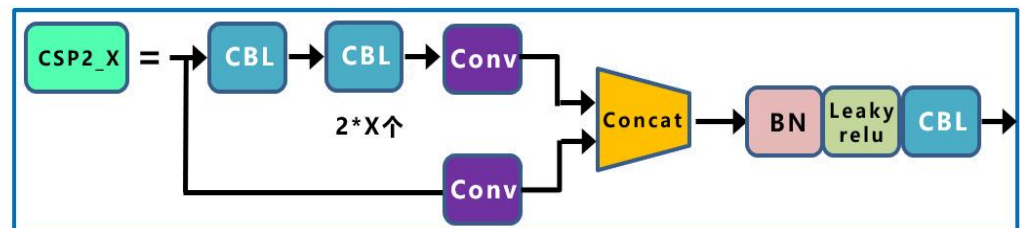
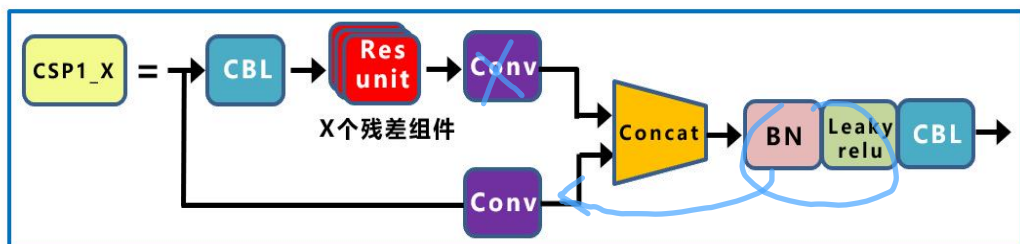
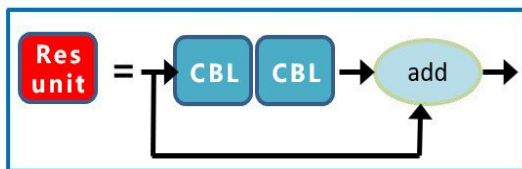
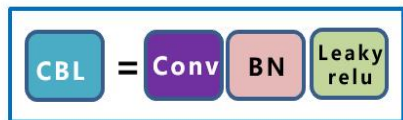
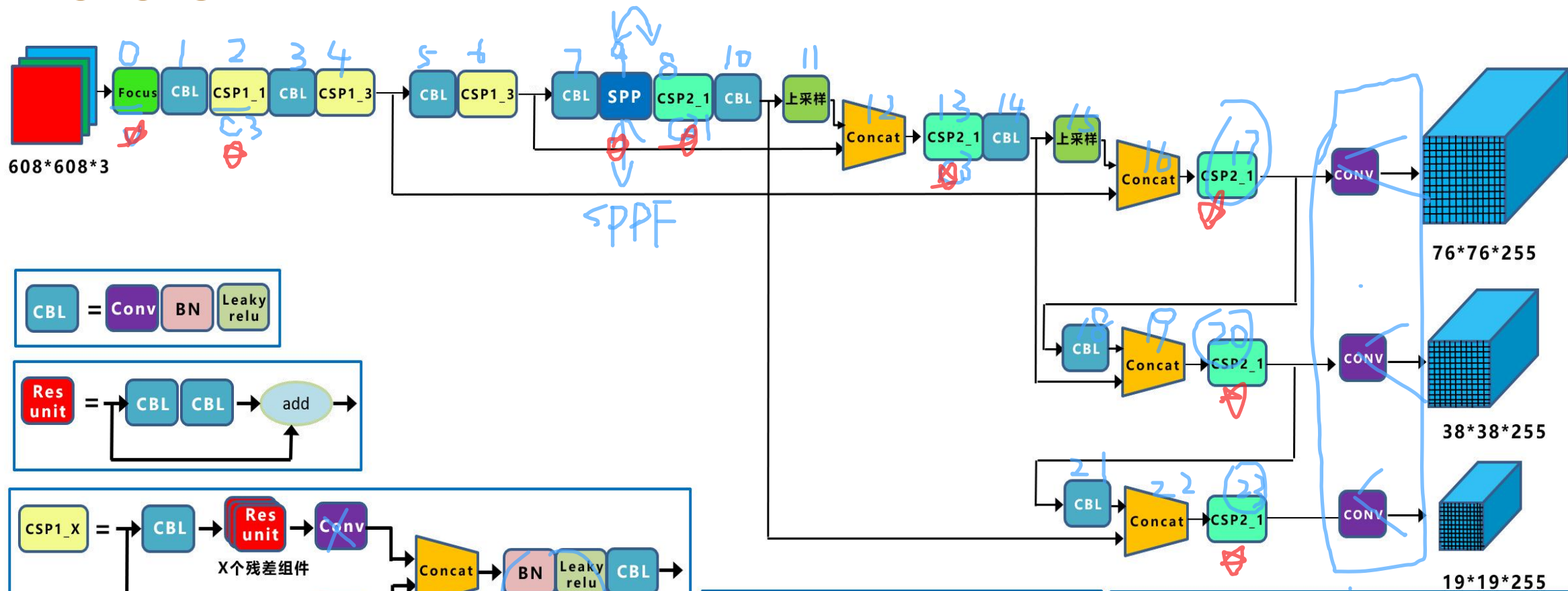
YOLO v5中的改进点(刚提出时候的结构):

- ⌚ 输入端: 数据增强(**Mosaic Augmentation**、MixUp、CutMix、Blurring、Label Smoothing等)、**自适应锚框计算、自适应的图像缩放**;
- ⌚ BackBone骨干网络: **Focus结构、CSP+DarkNet**等;
- ⌚ Neck: **FPN+PAN**、SPP等
- ⌚ Prediction: 更换边框回归损失函数(GIoU_LOSS);
- ⌚ 网站:
<https://github.com/ultralytics/yolov5>
<https://docs.ultralytics.com/zh/yolov5/>

YOLO v4



YOLOv5



YOLOv5_输入端

- ⌚ Mosaic数据增强

- ⌚ 自适应锚框计算

 - ⌚ YOLOv5中将计算先验框高宽比的程序代码集成到整个训练过程中(聚类代码)。

- ⌚ 自适应图片缩放

 - ⌚ 一般情况下，我们会将图像缩放到常用尺寸，然后在进行后续网络的执行；但是如果填充的较多的话，则会**存在较多的冗余信息，影响推理速度**。在YOLO v5中做了自适应的处理，以希望添加最少像素的黑边。

 - ⌚ YOLO v5里面填充的边是灰色，并且仅在推理预测的时候添加。

YOLOv5



长*宽: 800*600

第一步: $\begin{cases} 416/800=0.52 \\ 416/600=0.69 \end{cases}$

长*宽: 416*416

YOLOv5



长*宽: 800*600

第二步: $\begin{cases} 800 * 0.52 = 416 \\ 600 * 0.52 = 312 \end{cases}$

长*宽: 416*312

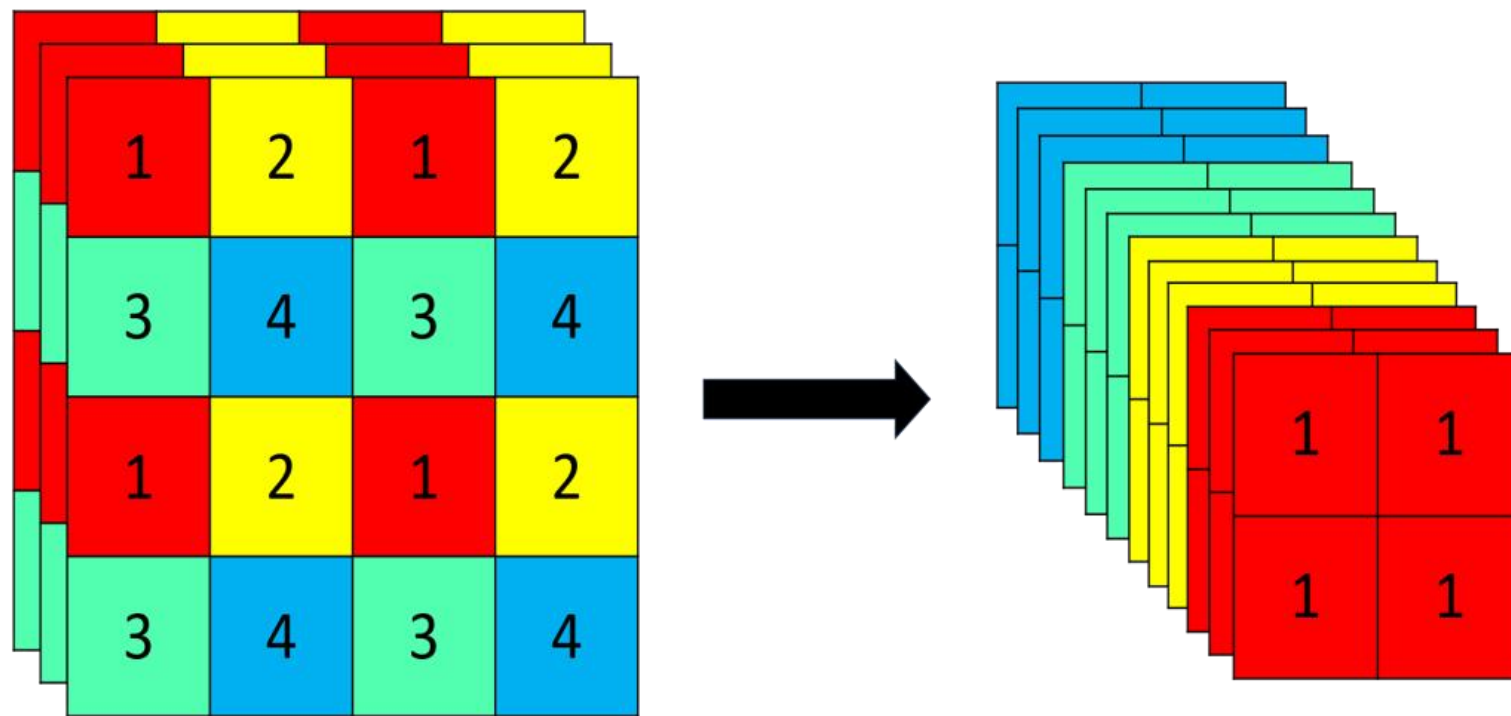
YOLOv5



长*宽: 800*600 第三步: $\begin{cases} 416-312=104 \\ \text{np.mod}(104,32)=8 \\ 8/2=4 \end{cases}$ 长*宽: 416*320 (312+8)

YOLOv5

🕒 Focus切片操作

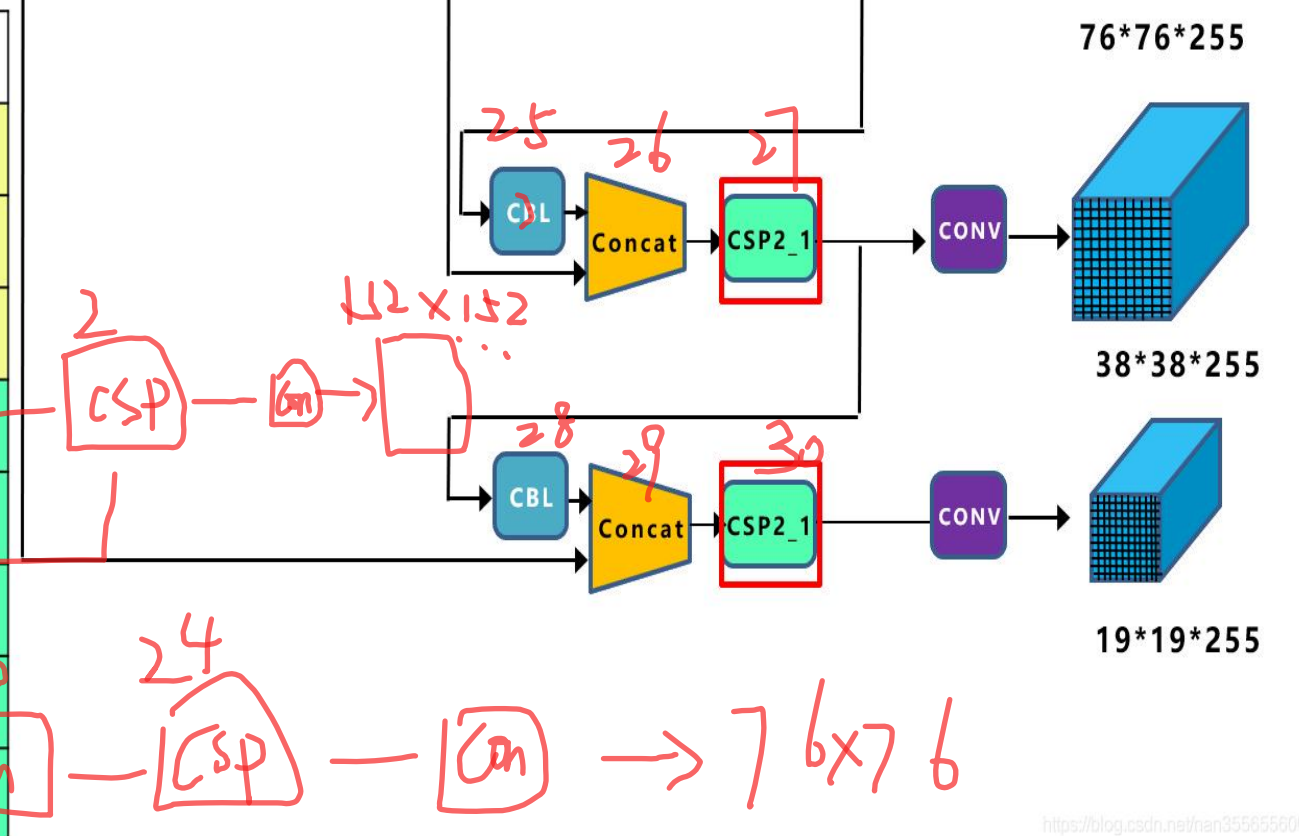


切片操作

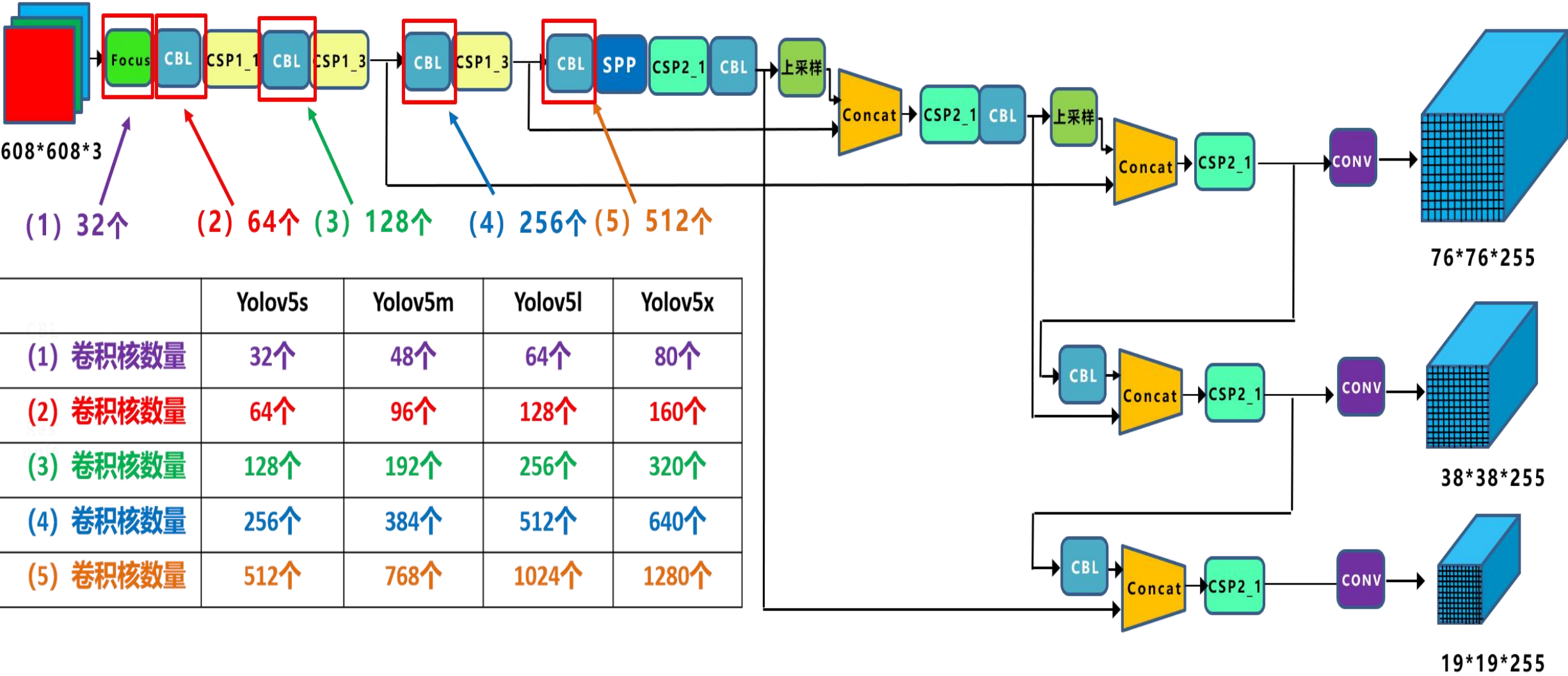
Diagram illustrating the YOLOv5 architecture flow:

- Input: $608 \times 608 \times 3$ image.
- Stages and Components:
 - Stage 0: Focus
 - Stage 1: CBL
 - Stage 2: CSP1_1 (highlighted)
 - Stage 3: CBL
 - Stage 4: CSP1_3 (highlighted)
 - Stage 5: CBL
 - Stage 6: CSP1_3 (highlighted)
 - Stage 7: CBL
 - Stage 8: SPP
 - Stage 9: CSP2_1 (highlighted)
 - Stage 10: CBL
 - Stage 11: 上采样 (Upsample)
 - Stage 12: Concat (Concatenation)
 - Stage 13: CSP2_1 (highlighted)
 - Stage 14: CBL
 - Stage 15: 上采样 (Upsample)
 - Stage 16: Concat (Concatenation)
 - Stage 17: CSP2_1 (highlighted)
 - Final Layer: CONV
- Connections: The diagram shows skip connections from the output of Stage 4 to Stage 12, and from Stage 10 to Stage 16.

	Yolov5s	Yolov5m	Yolov5l	Yolov5x
第一个CSP1	CSP1_1	CSP1_2	CSP1_3	CSP1_4
第二个CSP1	CSP1_3	CSP1_6	CSP1_9	CSP1_12
第三个CSP1	CSP1_3	CSP1_6	CSP1_9	CSP1_12
第一个CSP2	CSP2_1	CSP2_2	CSP2_3	CSP2_4
第二个CSP2	CSP2_1	CSP2_2	CSP2_3	CSP2_4
第三个CSP2	CSP2_1	CSP2_2	CSP2_3	CSP2_4
第四个CSP2	CSP2_1	CSP2_2	CSP2_3	CSP2_4
第五个CSP2	CSP2_1	CSP2_2	CSP2_3	CSP2_4



YOLOv5



YOLOv5

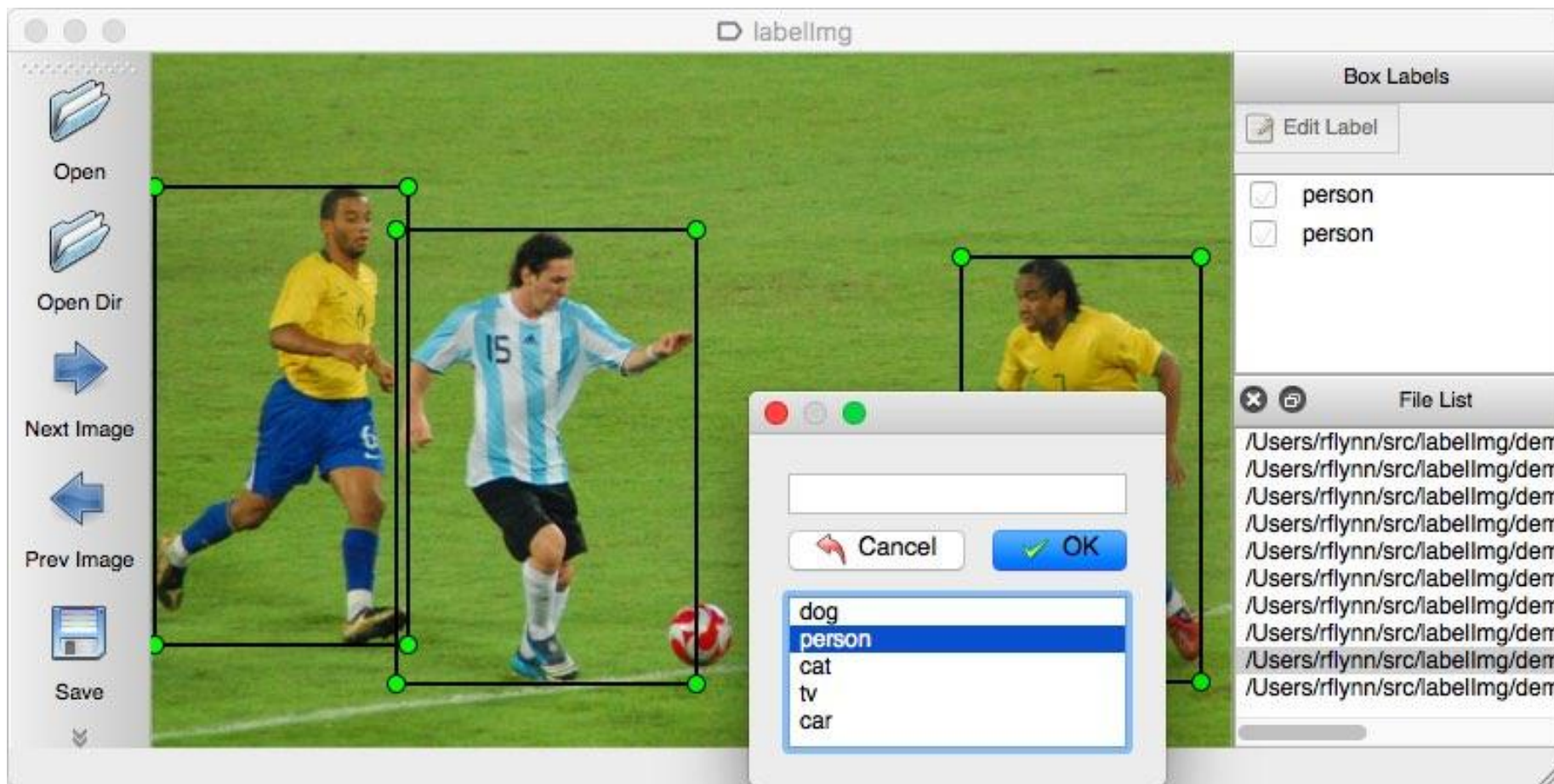
⌚ 代码: <https://github.com/ultralytics/yolov5>

⌚ 企业里面使用YOLOv5实现项目业务:

- ⌚ -1. 熟悉YOLOv5的现有代码;
- ⌚ -2. 将你的数据转换成代码要求的数据格式;
- ⌚ -3. 训练、评估;
- ⌚ -4. 调控、更改....

labelimg

- ⌚ 功能：一种进行目标检测的标注工具；
- ⌚ 官网： <https://github.com/heartexlabs/labelImg>



git

⌚ 一种常用的开发代码管理协同工具

⌚ 官网: <https://git-scm.com/>

⌚ 参考网站: <https://www.runoob.com/git/git-tutorial.html>

⌚ 添加ssh key: 命令行执行ssh-keygen -t rsa进行生成公钥, 然后将公钥copy到github或者gitee网站上进行设置即可。

```
(base) C:\Users\gerry_17578261252713>ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (C:\Users\gerry_17578261252713/.ssh/id_rsa):
Created directory 'C:\Users\gerry_17578261252713/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\gerry_17578261252713/.ssh/id_rsa.
Your public key has been saved in C:\Users\gerry_17578261252713/.ssh/id_rsa.pub.
The key fingerprint is:
SHA256:yP268adxJ3uVQjpERVkmCKVhUKnh1M751PSMdb95Kmw ecd\gerry_17578261252713@hz6mxclj8y80rf4
The key's randomart image is:
+---[RSA 2048]---+
|                 |
|  o*+=+=.       |
|   +o= o o      |
|  =.*.* . .     |
| . *. = + .     |
| S + o   +      |
|  o + . +o      |
| . o.o +. +     |
|   +Eoo+.       |
|  o+++. .       |
+---[SHA256]-----+
```

THANKS!